UNIVERSIDAD PERUANA CAYETANO HEREDIA

FACULTAD DE CIENCIAS E INGENIERÍA

Laboratorio 12

SISTEMAS OPERATIVOS



Grupo 4

Integrantes:

Hernandez Lopez, Justin Brayton

Rebatta Jeri, Patricia del Pilar

Medina Rojas, Martha Sofía

Vega Sanchez. Jimena Natalia

Profesor:
Roberto Eduardo Burga Montañez

Capítulo 1: Seguridad y Protección – Amenazas y Objetivos de Seguridad

Actividad 1: Identificación de permisos de archivos

Objetivo

Reconocer riesgos de seguridad asociados a permisos incorrectos.

Procedimiento

```
mkdir SeguridadLinux
cd SeguridadLinux

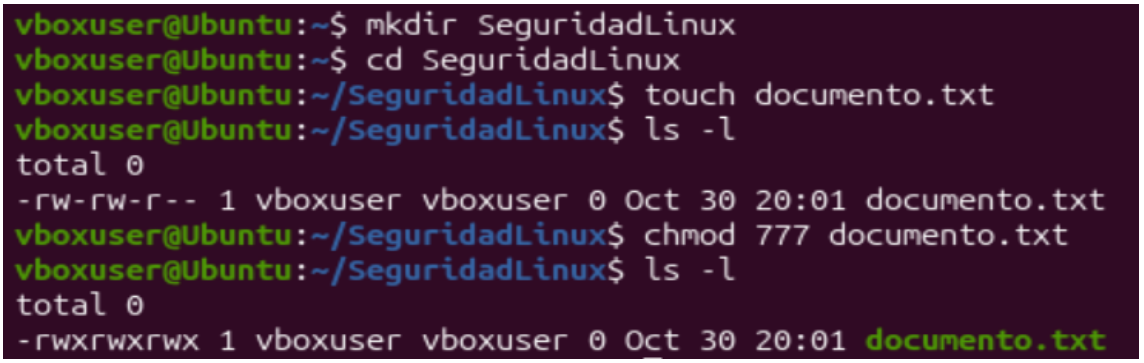
touch documento.txt
ls -l
```

Modificar permisos:

```
chmod 777 documento.txt
ls -l
```

Preguntas

- ¿Qué riesgo implica otorgar permisos 777?
- ¿Qué usuarios pueden modificar el archivo?
- ¿Cuál sería una configuración más segura?



1. ¿Qué riesgo implica otorgar permisos 777?

   Otorgar permisos 777 representa un riesgo de seguridad porque permite que cualquier usuario pueda leer, modificar y ejecutar el archivo. Esto puede provocar modificaciones no autorizadas, pérdida de información o acciones maliciosas.

2. ¿Qué usuarios pueden modificar el archivo?

   Todos los usuarios pueden mostrar el archivo, incluyendo:
   - El propietario del archivo.

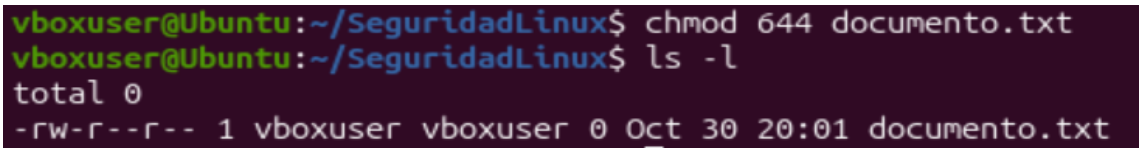
- Los usuarios que pertenecen al grupo.
- Los demás usuarios del sistema.

3. ¿Cuál sería una configuración más segura?

   Una configuración más segura sería utilizar el permiso 664: **chmod 664 document.txt**.

   Esto significa:

   - Propietario: puede leer y modificar
   - Grupo: solamente puede leer.
   - Otros usuarios: solamente pueden leer.



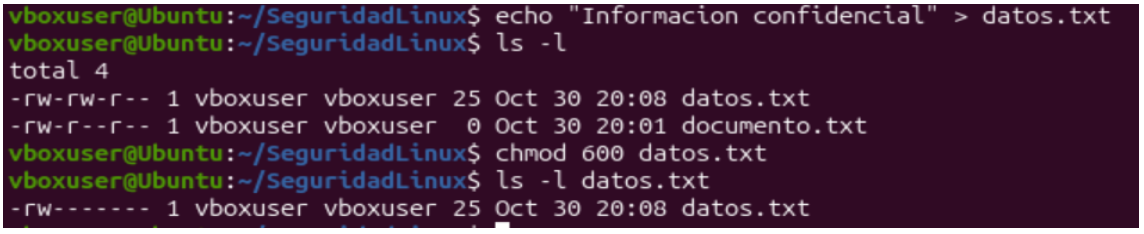
Actividad 2: Protección de información

Crear un archivo confidencial:

```
echo "Información confidencial" > datos.txt
chmod 600 datos.txt
ls -l datos.txt
```

Evidencia

Captura de pantalla mostrando los permisos.



Se creó un archivo llamado **datos.txt** que contiene información confidencial. Posteriormente, se modificaron sus permisos utilizando el comando **chmod 600**, permitiendo que únicamente el propietario pueda leer y modificar el archivo. De esta manera, se protege la información y se evita que otros usuarios puedan acceder a ella.

Capítulo 2: Intentos de Acceso No Autorizado

Actividad 1: Monitoreo de intentos de acceso

Visualizar registros de autenticación:

Ubuntu/Debian:

```
sudo cat /var/log/auth.log
```

CentOS/RHEL:

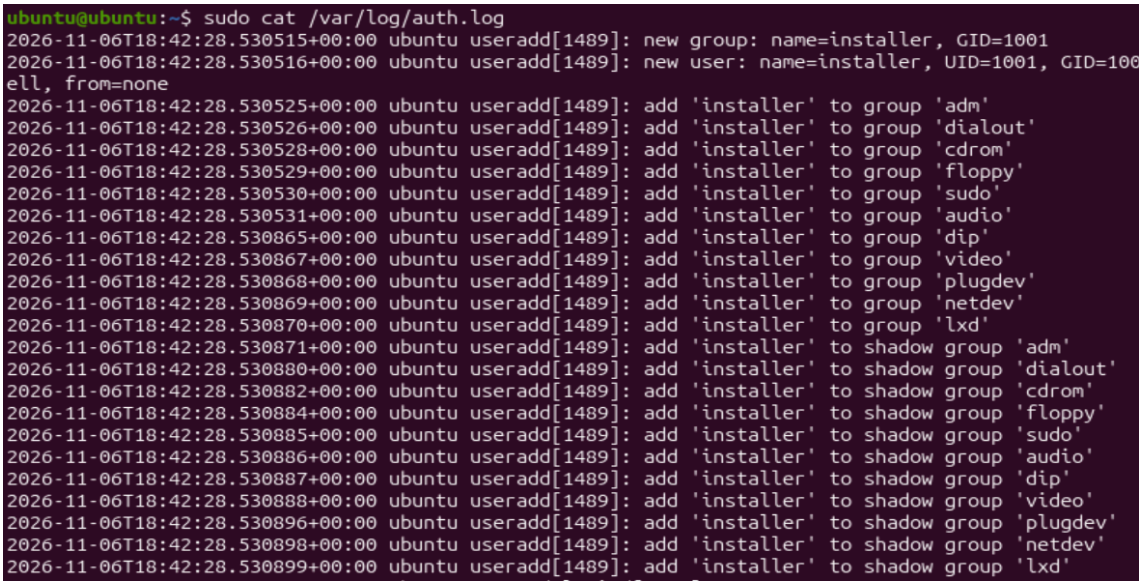
```
sudo cat /var/log/secure
```

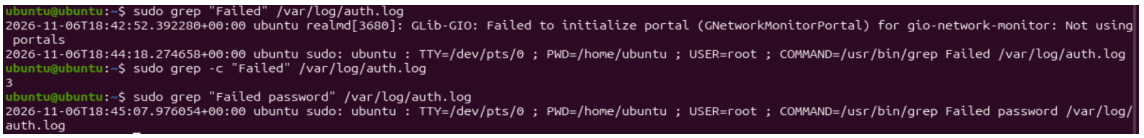
Buscar intentos fallidos:

```
sudo grep "Failed" /var/log/auth.log
```

Preguntas

- ¿Cuántos intentos fallidos se registraron?
- ¿Desde qué usuario se realizaron?





1. ¿Cuántos intentos fallidos se registraron?

   Se registraron 3 intentos fallidos de autenticación, según los registros del sistema.

2. ¿Desde qué usuario se realizaron?

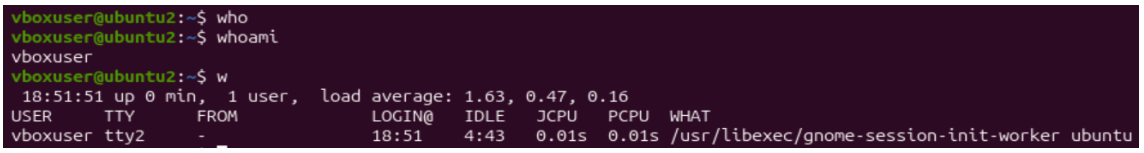
Los intentos fallidos se realizaron utilizando el siguiente nombre de usuario: vboxuser.

Actividad 2: Verificación de usuarios conectados

```
who
w
```

Evidencia

Registrar usuarios activos en el sistema.



Se ejecutó el comando **who** para identificar los usuarios conectados. El comando **whoami** permitió comprobar que el usuario actual del sistema es **ubuntu.** Sin embargo, **who** no mostró sesiones registradas.

Capítulo 3: Políticas y Mecanismos de Seguridad

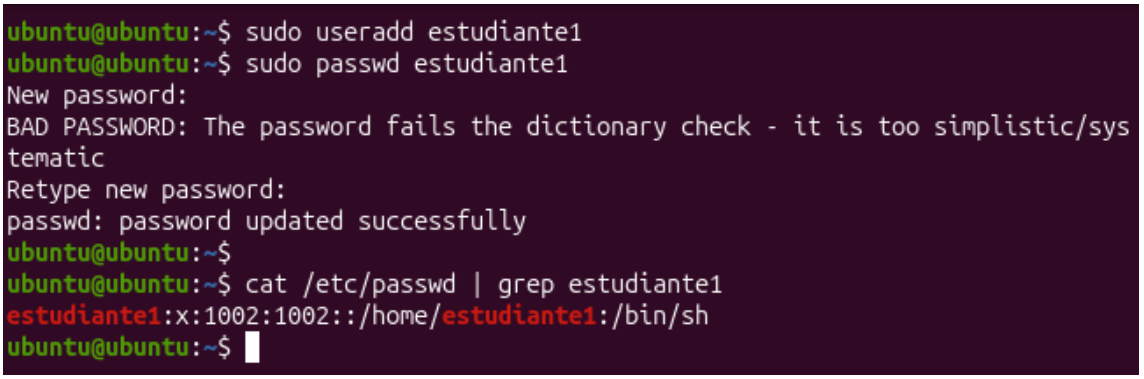
Actividad 1: Creación de usuarios

Crear un usuario:

```
sudo useradd estudiante1
sudo passwd estudiante1
```

Verificar:

```
cat /etc/passwd | grep estudiante1
```



Preguntas

- **¿Por qué es importante asignar cuentas individuales?**

  Porque permite identificar quién ingresa al sistema y lo que realiza. Además, ayuda a mejorar la seguridad, ya que cada persona tiene sus propias credenciales de acceso y permisos específicos.

  También facilita el control de las actividades realizadas evitando compartir contraseñas y permite establecer responsabilidades individuales en caso de presentarse algún problema de seguridad.

Actividad 2: Configuración de políticas de contraseña

Visualizar políticas:

```
sudo cat /etc/login.defs
```

Identificar:

```
PASS_MAX_DAYS
PASS_MIN_DAYS
PASS_WARN_AGE
```

Evidencia

```
#
# Password aging controls:
#
#       PASS_MAX_DAYS   Maximum number of days a password may be used.
#       PASS_MIN_DAYS   Minimum number of days allowed between password changes.
#       PASS_WARN_AGE   Number of days warning given before a password expires.
#
PASS_MAX_DAYS   99999
PASS_MIN_DAYS   0
PASS_WARN_AGE   7
```

Explicar la finalidad de cada parámetro.

A. **PASS_MAX_DAYS**
   - **Finalidad:** Establece el número máximo de días durante los cuales una contraseña puede utilizarse antes de que el sistema solicite cambiarla. En este caso, 99999.
   - **Importancia:** Ayuda a mantener actualizadas las contraseñas y reducir el riesgo de que una contraseña comprometida siga utilizándose durante mucho tiempo.

B. **PASS_MIN_DAYS**

- **Finalidad**: Establece el número mínimo de días que deben transcurrir antes de que un usuario pueda cambiar su contraseña nuevamente. En este caso es 0.

- **Importancia:** Evita que los usuarios cambien repetidamente sus contraseñas para regresar rápidamente a una contraseña anterior.

C. **PASS_WARN_AGE**
   - **Finalidad:** Indica cuántos días antes de que expire una contraseña el sistema debe comenzar a advertir al usuario que necesita cambiarla. En este caso, 7.

   - **Importancia:** Permite que los usuarios tengan tiempo suficiente para actualizar su contraseña antes de que expire.

Capítulo 4: Validación

Actividad 1: Verificación de integridad mediante hash

Crear archivo:

```
echo "Sistema Operativo" > archivo.txt
```

Generar hash:

```
sha256sum archivo.txt
```

Modificar archivo:

```
echo "Modificado" >> archivo.txt
sha256sum archivo.txt
```

Preguntas

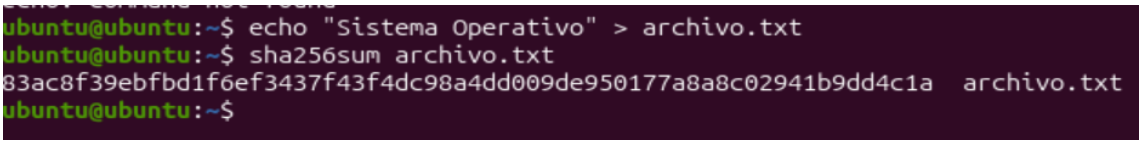
- **¿Cambió el hash?**

  Sí, el hash cambió porque se agregó nuevo contenido al archivo.

- **¿Qué demuestra este resultado?**

  Demuestra que el algoritmo SHA-256 permite detectar modificaciones en los archivos. Cuando cambia el contenido, se genera un hash diferente, lo que permite verificar su integridad.

- Evidencia 1



- Evidencia 2



Actividad 2: Validación de usuarios

Verificar información del usuario actual:

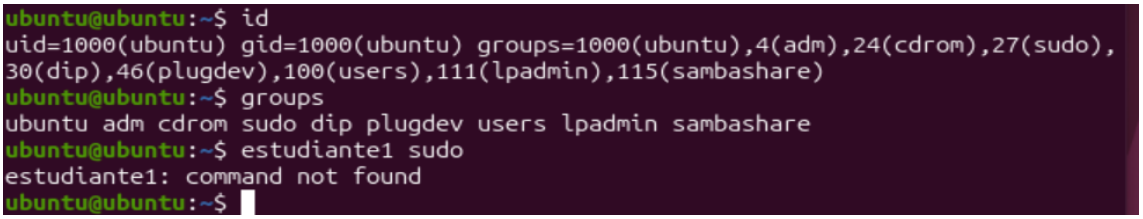
```
id
groups
```

Evidencia

Describir los grupos a los que pertenece el usuario.

Descripción de los grupos a los que pertenece el usuario:

El comando `id` permitió identificar el UID, el GID y los grupos asociados al usuario actual. Asimismo, mediante el comando `groups` se verificaron los grupos de pertenencia. Estos grupos determinan los permisos y privilegios que puede tener el usuario dentro del sistema operativo.



Los datos indican:

- `uid`: identificador del usuario.
- `gid`: identificador del grupo principal.
- `groups`: grupos a los que pertenece el usuario.

Significa que:

- `estudiante1`: es el grupo principal.
- `sudo`: permite ejecutar comandos administrativos mediante `sudo`.

Capítulo 5: Protección y Control de Acceso

Actividad 1: Gestión de grupos

Crear grupo:

```
sudo groupadd seguridad
```

Agregar usuario:

```
sudo usermod -aG seguridad estudiante1
```

Verificar:

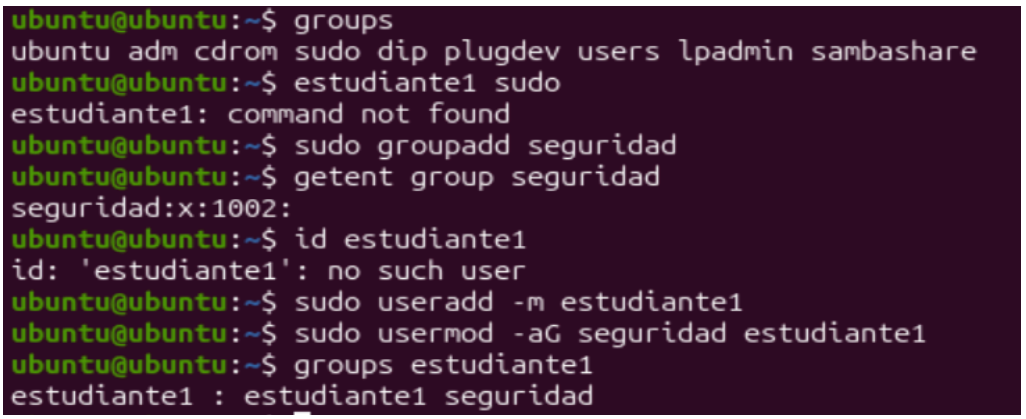
```
groups estudiante1
```

Preguntas

- ¿Qué ventajas ofrece el control por grupos?

El control por grupos permite administrar los permisos de varios usuarios de manera centralizada. Facilita la asignación de privilegios, evita configurar individualmente cada cuenta y mejora la organización y seguridad del sistema operativo.

Evidencia 1



Explicación:

- usermod: modifica la configuración de un usuario.
- -a: agrega sin eliminar otros grupos.
- -G: especifica el grupo secundario.
- seguridad: grupo al que se agregará.
- estudiante1: usuario que se modificará.

Actividad 2: Uso de ACL

Instalar ACL:

```
sudo apt install acl
```

Crear archivo:

```
touch informe.txt
```

Asignar permisos:

```
setfacl -m u:estudiante1:rwx informe.txt
```

Verificar:

```
getfacl informe.txt
```

Evidencia

Mostrar permisos ACL configurados.

Objetivo: Asignar permisos específicos a un usuario mediante listas de control de acceso (ACL).

Paso 1. Instalar ACL

Paso 2. Crear el archivo

Esto crea un archivo vacío llamado informe.txt.

Paso 3. Asignar permisos ACL

Significado:

- setfacl: configura permisos ACL.
- -m: modifica los permisos.
- u: indica que se asignarán a un usuario.
- estudiante1: usuario que recibirá los permisos
- rwx: lectura, escritura y ejecución.

```
vboxuser@ubuntuw:~$ sudo apt update
[sudo: authenticate] Password:
Hit:1 http://pe.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://pe.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://pe.archive.ubuntu.com/ubuntu resolute-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
181 packages can be upgraded. Run 'apt list --upgradable' to see them.
vboxuser@ubuntuw:~$ sudo apt install acl
acl is already the newest version (2.3.2-2).
acl set to manually installed.
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-14              linux-main-modules-zfs-7.0.0-14-generic  linux-tools-7.0.0-14-generic
  linux-headers-7.0.0-14-generic      linux-modules-7.0.0-14-generic
  linux-image-unsigned-7.0.0-14-generic  linux-tools-7.0.0-14
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 181
vboxuser@ubuntuw:~$ touch informe.txt
vboxuser@ubuntuw:~$ sudo setfacl -m u:estudiante1:rwx informe.txt
setfacl: Option -m: Invalid argument near character 3
```

Paso 4. Verificar los permisos

```
vboxuser@ubuntuw:~$ getfacl informe.txt
# file: informe.txt
# owner: vboxuser
# group: vboxuser
user::rw-
group::rw-
other::r--
```

Parte importante:

`user:estudiante1:rwx`

Esto demuestra que el usuario estudiante1 tiene permisos de lectura, escritura y ejecución sobre el archivo.

Se configuraron permisos ACL mediante el comando setfacl, otorgando al usuario estudiante1 permisos de lectura, escritura y ejecución sobre el archivo informe.txt.

Posteriormente, se utilizó getfacl para comprobar que los permisos fueron asignados correctamente.

Capítulo 6: Modelos Formales de Protección

Actividad 1: Aplicación del Principio de Mínimo Privilegio

Crear usuario:

```
sudo useradd auditor
```

Asignar únicamente permisos de lectura:

```
chmod 444 datos.txt
```

Probar acceso con:

```
su auditor
```

Intentar modificar:

```
echo "Prueba" >> datos.txt
```

Preguntas

- ¿Se pudo modificar el archivo?
- ¿Qué principio de seguridad se aplicó?

Objetivo: Comprobar que un usuario tenga únicamente los permisos necesarios para acceder a un archivo.

Paso 1. Crear el archivo datos.txt

Paso 2. Crear el usuario auditor

Esto crea el usuario auditor y su directorio personal.

Paso 3. Asignar una contraseña al usuario

Escribe una contraseña y confírmala.

La contraseña no aparecerá mientras la escribes. Es normal en Linux.

Paso 4. Asignar únicamente permisos de lectura

El valor 4 otorga exclusivamente permisos de lectura (r--). Al aplicarlo a Propietario, Grupo y Otros (444), el archivo queda configurado como solo lectura para todos los usuarios del sistema.

Ningún usuario tendrá permisos de escritura ni ejecución sobre el archivo.

Paso 5. Verificar los permisos

Paso 6. Iniciar sesión como auditor

Introduce la contraseña que configuraste anteriormente.

Paso 7. Intentar modificar el archivo

El sistema debería mostrar: Permission denied

Esto sucede porque el usuario auditor no tiene permisos de escritura.

Paso 8. Regresar al usuario anterior

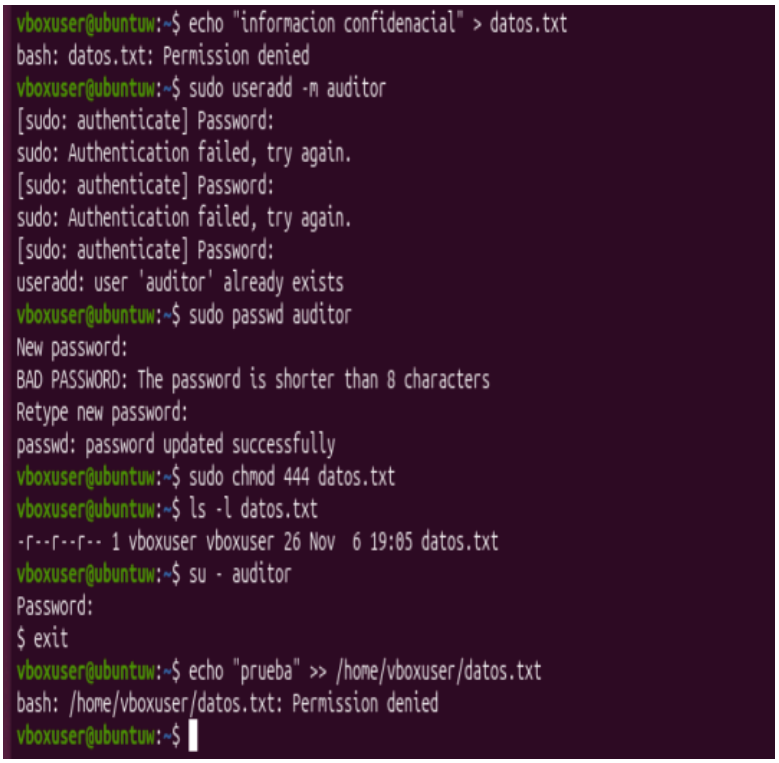
Paso 9. Responder las preguntas

¿Se pudo modificar el archivo?

No, el usuario auditor no pudo modificar el archivo porque únicamente tiene permisos de lectura.

¿Qué principio de seguridad se aplicó?

Se aplicó el principio de mínimo privilegio, que establece que cada usuario debe tener únicamente los permisos necesarios para realizar sus actividades, evitando accesos o modificaciones no autorizadas.



Actividad 2: Análisis de privilegios

Identificar procesos ejecutados como root:

```
ps -ef | grep root
```

Evidencia

Seleccionar tres procesos y explicar su función.

Objetivo: Identificar procesos que se ejecutan con privilegios administrativos (root) en Linux.

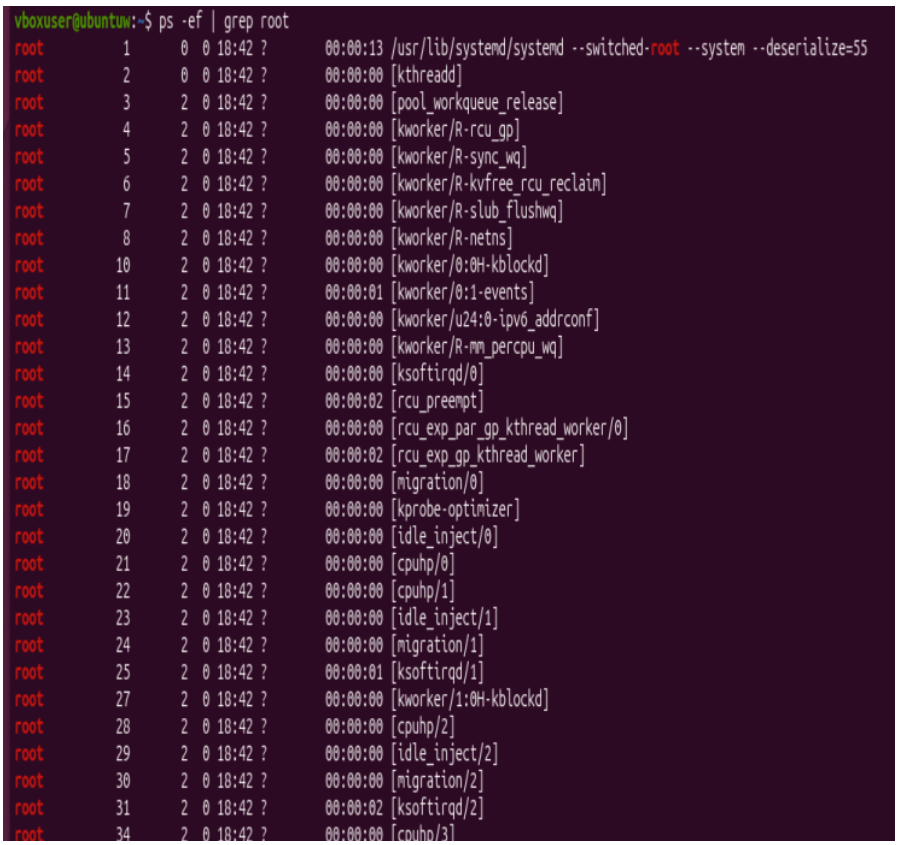
Paso 1. Abrir la terminal

Paso 2. Ejecutar el comando solicitado

Este comando permite visualizar procesos relacionados con el usuario `root`.

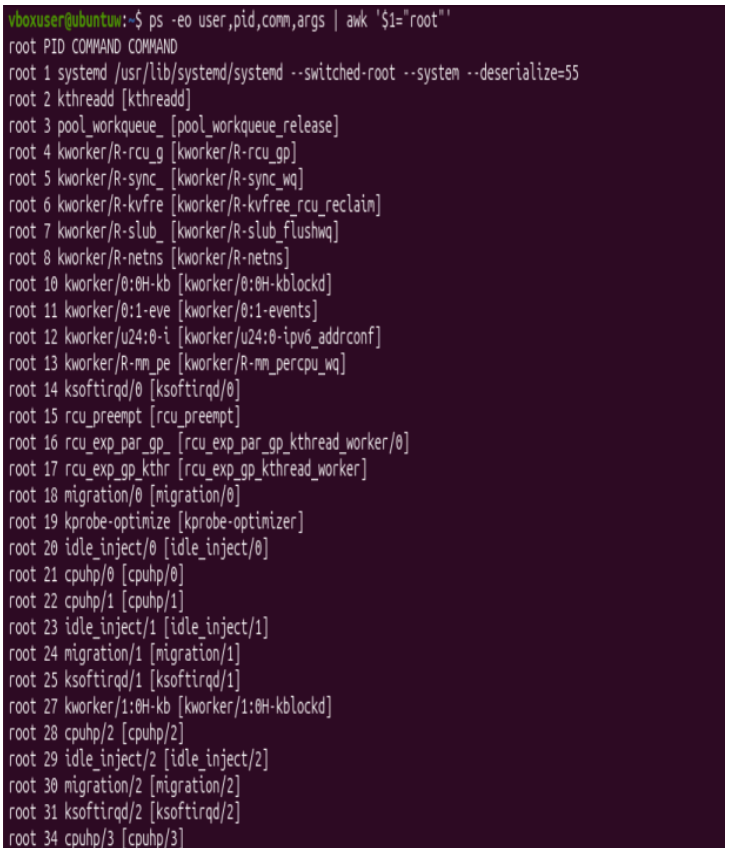
Explicación:

- ps: muestra los procesos activos.
- -e: muestra todos los procesos.
- -f: presenta información detallada.
- grep root: filtra las líneas que contienen la palabra root.



Paso 3. Identificar tres procesos

En el resultado busca tres procesos cuyo propietario sea `root`.



Paso 4. Comprender los procesos

Algunos procesos que podríamos encontrar son:

1. systemd

Administración del sistema.- Es el proceso inicial del sistema Linux. Se encarga de iniciar y administrar servicios y otros procesos.

2. sshd

Conexiones remotas.-Es el servicio encargado de gestionar conexiones remotas seguras mediante el protocolo SSH.

3. cron

Tareas programadas.-Es un servicio que permite ejecutar automáticamente tareas programadas en determinados horarios.

Se utilizó el comando ps -ef | grep root para identificar los procesos ejecutados con privilegios administrativos en Linux. Se analizaron tres procesos, identificando sus funciones dentro del

sistema operativo. Se comprobó que el usuario root posee privilegios elevados para administrar servicios y recursos del sistema.

Actividad Integradora Final

Caso práctico

Una empresa detecta intentos de acceso no autorizado a un servidor Linux.

Los estudiantes deberán:

1. Crear usuarios y grupos.
2. Configurar permisos seguros.
3. Implementar ACL.
4. Revisar registros de autenticación.
5. Generar hashes para validar integridad.
6. Elaborar un informe con evidencias de comandos y resultados obtenidos.

Entregable

- Capturas de pantalla.
- Comandos ejecutados.
- Análisis de resultados.
- Conclusiones sobre la seguridad implementada.

Informe de actividad integradora final

1. INTRODUCCIÓN

En la presente actividad integradora se desarrolló un caso práctico relacionado con la seguridad informática en un servidor Linux. El escenario plantea que una empresa ha detectado intentos de acceso no autorizado a su servidor, por lo que resulta necesario implementar diferentes mecanismos de protección.

Para ello, se realizaron procedimientos de administración de usuarios y grupos, configuración de permisos, implementación de listas de control de acceso (ACL), revisión de registros de autenticación y generación de hashes para verificar la integridad de los archivos.

El propósito de estas actividades es fortalecer la seguridad del servidor y proteger la información de la empresa frente a accesos y modificaciones no autorizadas.

2. OBJETIVO GENERAL

Implementar mecanismos de seguridad en un servidor Linux mediante la administración de usuarios, grupos, permisos, ACL, revisión de registros de autenticación y generación de hashes para garantizar la protección e integridad de la información.

3. DESARROLLO DE LA ACTIVIDAD

Creación de usuarios y grupos:

Descripción:

En esta primera etapa se crearon diferentes usuarios y grupos para organizar el acceso al servidor Linux según las responsabilidades de cada trabajador.

Se crearon los usuarios gerente, empleado y auditor. También se crearon los grupos administración y auditoría.

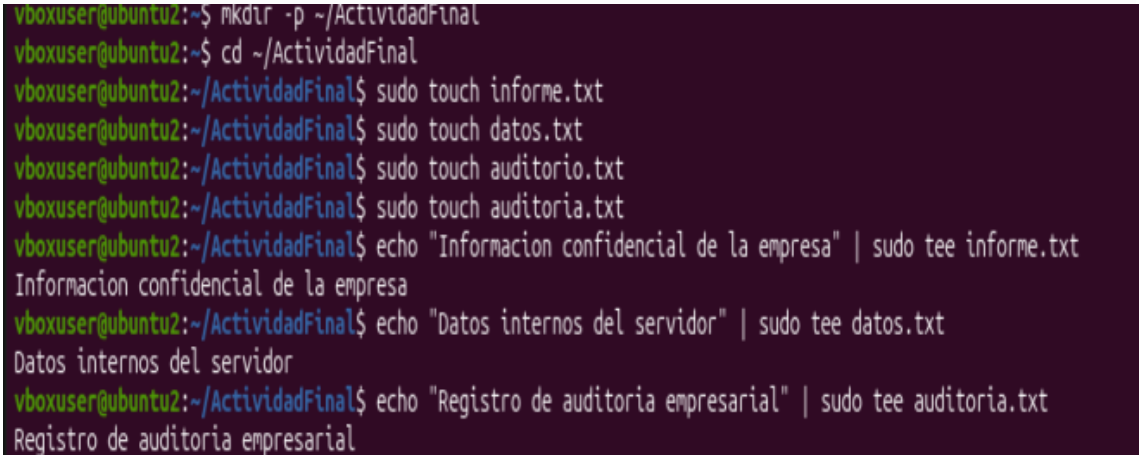
Comandos utilizados:

- sudo groupadd administración
- sudo groupadd auditoría
- sudo useradd -m -s /bin/bash gerente
- sudo useradd -m -s /bin/bash empleado
- sudo useradd -m -s /bin/bash auditor
- sudo usermod -aG administracion gerente
- sudo usermod -aG administracion empleado
- sudo usermod -aG auditoria auditor

Análisis de resultados:

Se logró crear correctamente los usuarios y grupos, permitiendo organizar las cuentas y establecer una estructura de administración de accesos dentro del servidor.

Evidencia 1: Capturas de pantalla de la creación de usuarios y grupos.



```
vboxuser@ubuntu2:~/ActividadFinal$ sudo groupadd administracion
vboxuser@ubuntu2:~/ActividadFinal$ sudo groupadd auditoria
vboxuser@ubuntu2:~/ActividadFinal$ sudo useradd -m -s /bin/bash gerente
vboxuser@ubuntu2:~/ActividadFinal$ sudo useradd -m -s /bin/bash empleado
vboxuser@ubuntu2:~/ActividadFinal$ sudo useradd -m -s /bin/bash auditor
useradd: user 'auditor' already exists
vboxuser@ubuntu2:~/ActividadFinal$ sudo passwd gerente
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
Sorry, passwords do not match.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
vboxuser@ubuntu2:~/ActividadFinal$ sudo passwd empleado
New password:
BAD PASSWORD: The password contains the user name in some form
Retype new password:
passwd: password updated successfully
vboxuser@ubuntu2:~/ActividadFinal$ sudo passwd auditor
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
```

```
vboxuser@ubuntu2:~/ActividadFinal$ sudo usermod -aG administracion gerente
vboxuser@ubuntu2:~/ActividadFinal$ sudo usermod -aG administracion empleado
vboxuser@ubuntu2:~/ActividadFinal$ sudo usermod -aG auditoria auditor
vboxuser@ubuntu2:~/ActividadFinal$ id gerente
uid=1002(gerente) gid=1004(gerente) groups=1004(gerente),1002(administracion)
vboxuser@ubuntu2:~/ActividadFinal$ id empleado
uid=1003(empleado) gid=1005(empleado) groups=1005(empleado),1002(administracion)
vboxuser@ubuntu2:~/ActividadFinal$ id auditor
uid=1001(auditor) gid=1001(auditor) groups=1001(auditor),1003(auditoria)
```

Configurar permisos seguros:

Descripción:

En esta etapa se configuraron los permisos de los archivos para controlar quién podía leer, escribir o acceder a la información almacenada.

Se utilizaron los comandos chown y chmod para asignar propietarios, grupos y permisos adecuados.

Comandos utilizados:

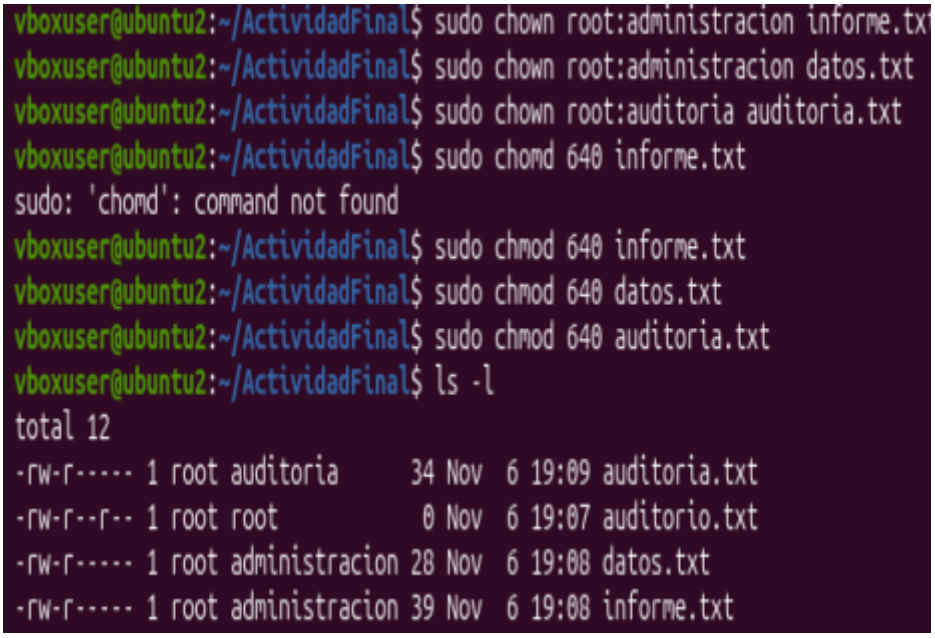
- sudo chown root:administracion informe.txt
- sudo chown root:administracion datos.txt
- sudo chown root:auditoria auditoria.txt
- sudo chmod 640 informe.txt
- sudo chmod 640 datos.txt
- sudo chmod 640 auditoria.txt
- ls -l

Análisis de resultados:

Se establecieron permisos que restringen el acceso a los archivos. El propietario tiene permisos de lectura y escritura, el grupo tiene permisos de lectura y los demás usuarios no tienen acceso.

Esta configuración ayuda a prevenir modificaciones y consultas no autorizadas.

Evidencia 2: Capturas de pantalla de los comandos y resultados de los permisos configurados.



Implementar ACL:

Descripción:

Se implementaron listas de control de acceso (ACL) para conceder permisos específicos a determinados usuarios sin modificar los permisos generales de los grupos.

Se asignaron permisos de lectura al usuario auditor sobre el archivo informe.txt y permisos de lectura y escritura al usuario gerente sobre datos.txt.

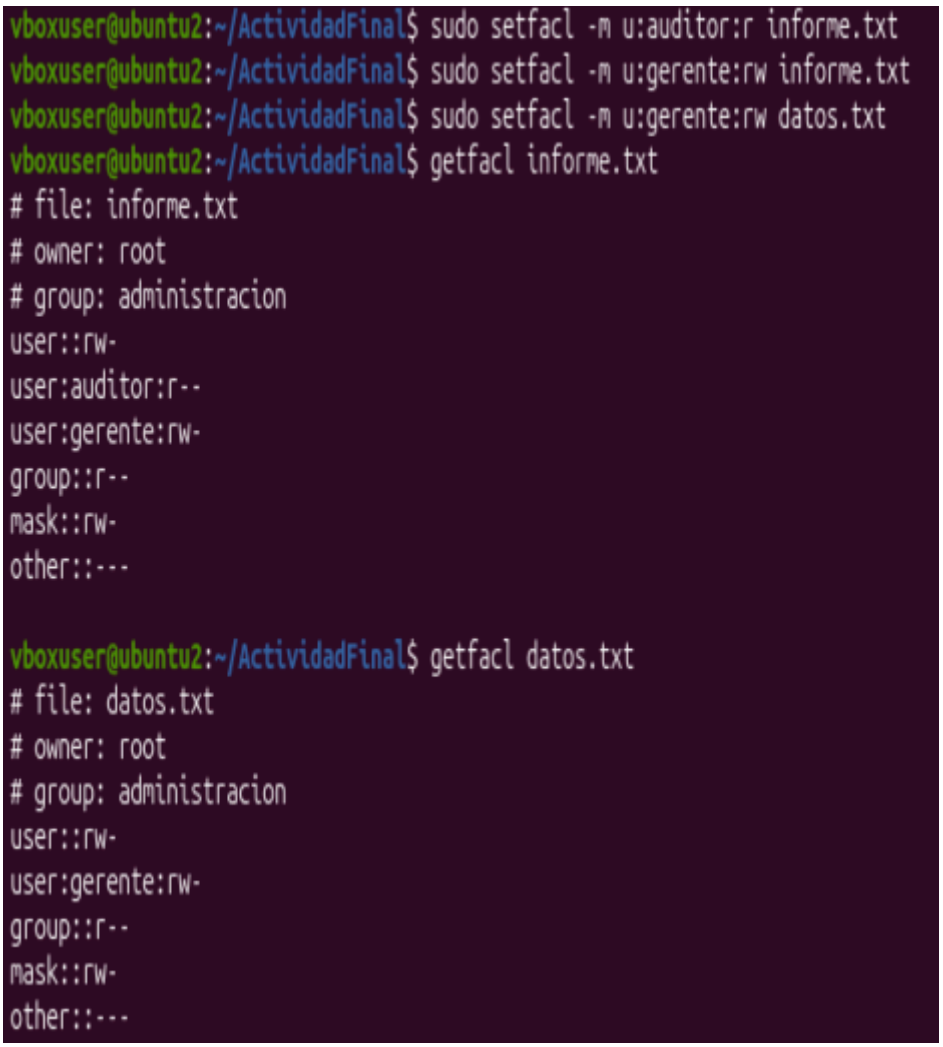
Comandos utilizados:

- sudo setfacl -m u:auditor:r informe.txt
- sudo setfacl -m u:gerente:rw datos.txt
- getfacl informe.txt
- getfacl datos.txt

Análisis de resultados:

Se comprobó que las ACL permiten establecer permisos personalizados para cada usuario. De esta manera, el auditor puede consultar información autorizada y el gerente puede modificar el archivo asignado.

Evidencia 3: Capturas de pantalla de la configuración y comprobación de las ACL.



Revisar registros de autenticación:

Descripción:

En esta etapa se revisaron los registros de autenticación del servidor Linux para identificar posibles intentos de acceso no autorizado.

Se consultaron los registros del sistema para buscar intentos de inicio de sesión fallidos y accesos aceptados.

Comandos utilizados:

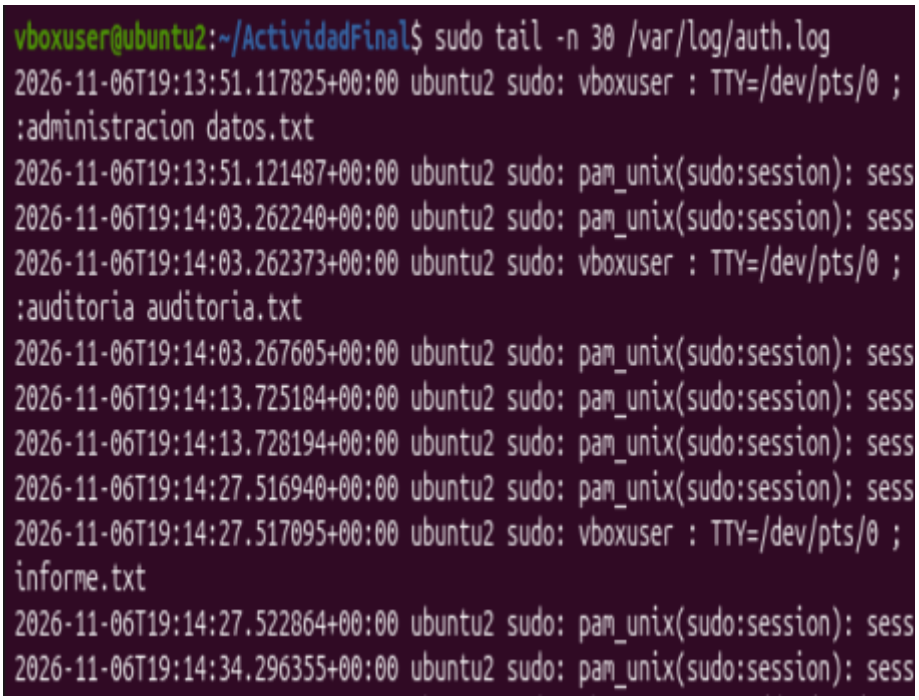
- sudo grep -i "failed" /var/log/auth.log
- sudo grep -i "accepted" /var/log/auth.log
- sudo journalctl --no-pager -n 40

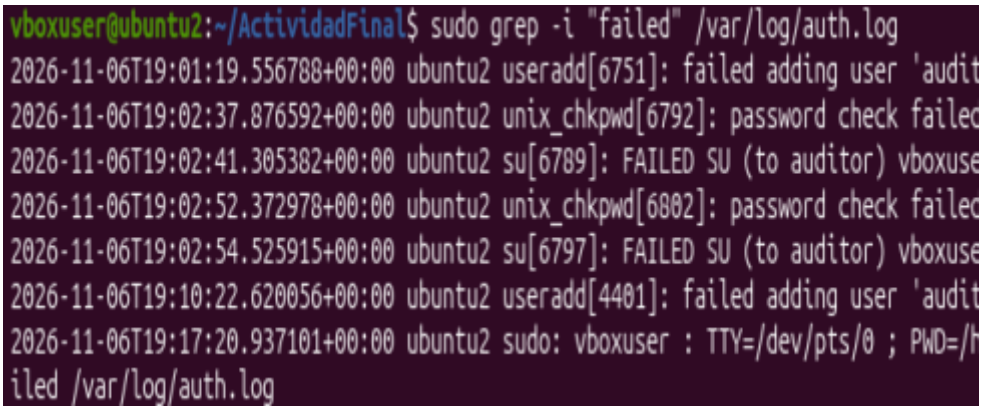
Análisis de resultados:

La revisión de los registros permitió consultar los eventos de autenticación del sistema, identificar los intentos fallidos cuando estaban registrados y observar los accesos aceptados.

Estos registros son importantes porque permiten supervisar la actividad de los usuarios y detectar posibles comportamientos sospechosos.

Evidencia 4: Capturas de pantalla de los registros de autenticación consultados.





```
vboxuser@ubuntu2:~/ActividadFinal$ sudo grep -i "accepted" /var/log/auth.log
2026-11-06T19:17:44.269800+00:00 ubuntu2 sudo: vboxuser : TTY=/dev/pts/0 ; PWD=/home/vboxuser/ActividadFinal ; USER=root ; COMMAND=/usr/bin/grep -i ac
cepted /var/log/auth.log
vboxuser@ubuntu2:~/ActividadFinal$ sudo journalctl -u ssh --no-pager -n 30
-- No entries --
vboxuser@ubuntu2:~/ActividadFinal$ sudo journalctl --no-pager | grep -i "authentication failure"
Nov 06 19:02:37 ubuntu2 su[6789]: pam_unix(su:auth): authentication failure; logname=vboxuser uid=1000 euid=0 tty=/dev/pts/0 ruser=vboxuser rhost=  us
er=auditor
Nov 06 19:02:52 ubuntu2 su[6797]: pam_unix(su:auth): authentication failure; logname=vboxuser uid=1000 euid=0 tty=/dev/pts/0 ruser=vboxuser rhost=  us
er=auditor
```

Generar hashes para validar la integridad:

Descripción:

En esta última etapa se utilizó el algoritmo SHA-256 para generar hashes de los archivos de prueba y comprobar si su contenido había sido modificado.

Primero se generaron y guardaron los hashes originales. Posteriormente, se modificó el archivo datos.txt y se realizó una nueva comparación.

Comandos utilizados:

- sudo sha256sum informe.txt datos.txt auditoria.txt
- sudo sha256sum informe.txt datos.txt auditoria.txt | sudo tee hashes_originales.txt
- echo "Modificacion de prueba" | sudo tee -a datos.txt
- sudo sha256sum -c hashes_originales.txt

Análisis de resultados:

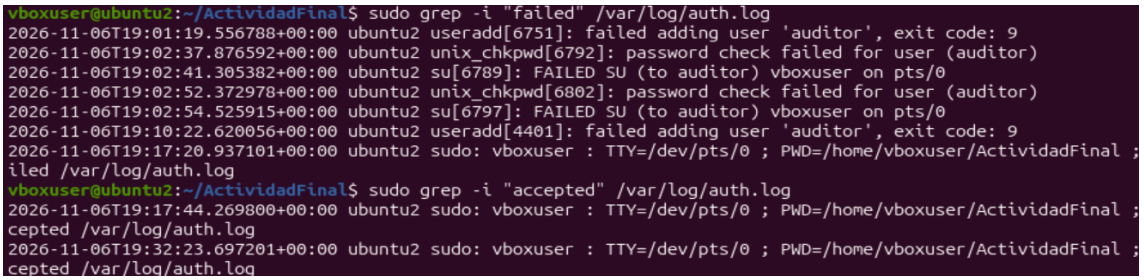
La comparación permitió comprobar que los archivos que conservaron su contenido original mostraron el resultado OK, mientras que el archivo datos.txt mostró FAILED debido a que había sido modificado.

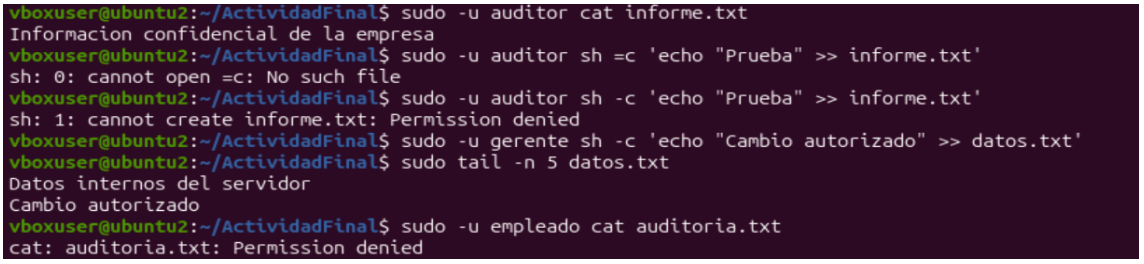
Esto demuestra que SHA-256 permite detectar cambios en el contenido de los archivos y contribuye a verificar la integridad de la información.

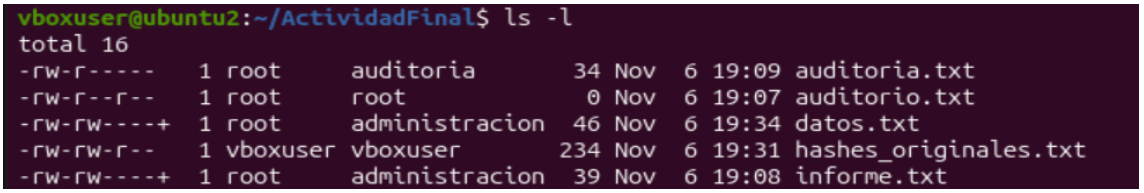
Evidencia 5: Capturas de pantalla de la generación y comparación de los hashes.

```
vboxuser@ubuntu2:~/ActividadFinal$ sudo sha256sum informe.txt datos.txt auditoria.txt
ec4098ea81276fb4bab625caefda9da75699a4ca7625d5ffaa623e313a02ed25  informe.txt
5b7677189df73781f9f97e54dd3e78d9763776b3bb569a030774f2169817940d  datos.txt
2aea40a71f71f174e092ef9154b861ee114a66dd2a356a2b1ac7ed3fbfe4b547  auditoria.txt
vboxuser@ubuntu2:~/ActividadFinal$ sudo sha256sum informe.txt datos.txt auditoria.txt |
ec4098ea81276fb4bab625caefda9da75699a4ca7625d5ffaa623e313a02ed25  informe.txt
5b7677189df73781f9f97e54dd3e78d9763776b3bb569a030774f2169817940d  datos.txt
2aea40a71f71f174e092ef9154b861ee114a66dd2a356a2b1ac7ed3fbfe4b547  auditoria.txt
```

```
vboxuser@ubuntu2:~/ActividadFinal$ sudo sha256sum informe.txt datos.txt auditoria.txt > hashes_originales.txt
vboxuser@ubuntu2:~/ActividadFinal$ cat hashes_originales.txt
ec4098ea81276fb4bab625caefda9da75699a4ca7625d5ffaa623e313a02ed25  informe.txt
5b7677189df73781f9f97e54dd3e78d9763776b3bb569a030774f2169817940d  datos.txt
2aea40a71f71f174e092ef9154b861ee114a66dd2a356a2b1ac7ed3fbfe4b547  auditoria.txt
```







4. ANÁLISIS GENERAL DE RESULTADOS

Durante el desarrollo de la actividad se aplicaron diferentes mecanismos de seguridad para proteger los archivos y controlar el acceso al servidor Linux.

La creación de usuarios y grupos permitió organizar las cuentas según sus funciones. La configuración de permisos estableció restricciones de lectura y escritura, mientras que las ACL permiten asignar autorizaciones específicas.

Por otro lado, la revisión de registros de autenticación permite consultar los eventos de acceso al sistema. Finalmente, la generación de hashes SHA-256 permitió detectar modificaciones realizadas en los archivos.

Los resultados obtenidos demuestran la importancia de combinar diferentes mecanismos de seguridad para proteger la información de una organización.

5. CONCLUSIONES

1. La creación de usuarios y grupos permite organizar los accesos al servidor y administrar las responsabilidades de cada cuenta.
2. La configuración de permisos mediante chmod ayuda a restringir el acceso a los archivos y prevenir modificaciones no autorizadas.
3. Las listas de control de acceso (ACL) permiten asignar permisos específicos a determinados usuarios, proporcionando mayor flexibilidad en la administración de la seguridad.
4. La revisión de los registros de autenticación permite supervisar los accesos al sistema e identificar posibles intentos de acceso no autorizado.

5. La utilización de hashes SHA-256 permite verificar la integridad de los archivos y detectar modificaciones en su contenido.

En conclusión, la actividad permitió comprender y aplicar diferentes mecanismos de protección en Linux, demostrando que una correcta administración de usuarios, permisos y registros contribuye a fortalecer la seguridad de un servidor empresarial.